

Esame OS Internals (Cabodi)

Data: 20/06/2022

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, TLB e EAT)

Contesto: Sia dato un sistema di memoria virtuale con paginazione, nel quale vengono indirizzati i Byte. Il sistema dispone di TLB (Translation Look-aside Buffer). La tabella delle pagine ("page-table") viene realizzata con uno schema a due livelli, nel quale un indirizzo logico di 64 bit viene suddiviso (da MSB a LSB) in 3 parti: p_1 , p_2 e d , p_2 ha 14 bit e d 13 bit. Non si utilizzano ulteriori strutture dati (quali tabelle di hash o inverted page table) per velocizzare gli accessi. La memoria virtuale viene gestita con paginazione a richiesta.

Si risponda alle seguenti domande:

-) Supponendo che la memoria RAM abbia tempo di accesso di 75 ns, si calcoli il TLB miss ratio (frequenza di miss), sapendo che, a causa dei TLB miss, si sa che $EAT_{PT} \geq 100$ ns, assumendo che il tempo di accesso alla TLB sia trascurabile. Si tratta di un lower bound o di un upper bound per la TLB miss ratio?
-) Si consideri ora la frequenza di page fault P_{PF} . Una valutazione sperimentale mostra che un page fault viene servito in 2 ms in media, e che il degrado di prestazioni (aumento di tempo di esecuzione) causato dai page fault è compreso tra 10% e 30%. Calcolare l'intervallo di valori di P_{PF} compatibile con la valutazione sperimentale, assumendo $EAT_{PT} = 100$ ns.
-) Una seconda valutazione sperimentale (una simulazione) viene fatta usando due stringhe di riferimento w_1 e w_2 aventi lunghezza, rispettivamente, $len(w_1) = 10^6$, $len(w_2) = 2 \cdot 10^6$ e probabilità di stringa p_1 e p_2 . Le simulazioni hanno generato 400 page fault in totale (100 su w_1 e 300 su w_2) e stimato una probabilità empirica $f = 0.00012$ (frequenza attesa) che avvenga un page fault nel sistema reale. Si calcolino i valori di p_1 e p_2 .

Domanda 2 (File System Unix-like e Inodes)

Contesto: Considerare un file system Unix-like, basato su inode, con 13 puntatori / indici (10 diretti, 1 singolo indiretto, 1 doppio indiretto e 1 triplo indiretto). I puntatori / indici hanno una dimensione di 32 bit e i blocchi del disco hanno una dimensione di 2 KB. Il file system risiede su una partizione del disco di 800 GB, che include sia blocchi di dato che blocchi di indice.

Si risponda alle seguenti domande:

-) Supponendo che tutti i metadati (eccetto i blocchi indice) abbiano dimensioni trascurabili, calcolare il numero massimo di file che il file system può ospitare, utilizzando l'indicizzazione indiretta doppia (N2) e l'indicizzazione indiretta tripla (N3).
-) Dato un file binario di dimensione 10240.5 KB, calcolare esattamente quanti blocchi indice e blocchi di dato occupa il file.

-) Considerare lo stesso file della domanda B, su cui viene chiamata l'operazione `lseek (fd, offset, SEEK_END)` per posizionare l'offset del file per la successiva operazione di lettura / scrittura (`SEEK_END` significa che l'offset viene calcolato dalla fine del file). Supponiamo che `fd` sia il descrittore di file associato al file (già aperto), e `offset = 0x00800000`.
1. Calcolare il numero di blocco logico (numerazione relativa ai blocchi del file, numerati a partire da 0) in corrispondenza del quale viene spostata la posizione (da `lseek`).
 2. Se il file utilizza un'indicizzazione indiretta singola (o doppia / tripla), calcolare quale riga del blocco di indici esterno contiene l'indice del blocco di dati (o l'indice del blocco di indici più interno).
-

Domanda 3 (Dispositivi di I/O, Polling e Interrupts)

Contesto: Un processo user vuole effettuare un input da un dispositivo di IO a caratteri (character device) mediante una strategia basata su polling del dispositivo. Per evitare un'attesa troppo lunga (nel loop di polling) da parte del driver del dispositivo, si chiede all'autore del programma eseguito dal processo user, di fare direttamente polling mediante un loop di lettura del registro di stato del dispositivo.

Si risponda alle seguenti domande:

-) E' possibile effettuare questa operazione? (se si, dire come, se no, dire perché).
-) Supponendo che il dispositivo sia associato alla tastiera e lo si gestisca in interrupt, ci si aspetta che venga generato un interrupt per ogni carattere ricevuto oppure un interrupt per ogni "invio" (cioè enter, fine-riga)? (Motivare).
-) Dato un altro dispositivo di IO, gestito a blocchi: E' possibile che una sola scrittura (mediante chiamata a `write()`) tenti di fare output di più di una coppia puntatore/dimensione?
-) Dato il driver che realizza la scrittura, supponendo che il sistema sia dotato di DMA controller, è possibile che venga eseguita "prima" una operazione di lettura e/o scrittura attivata successivamente a questa, da un altro processo? E dallo stesso processo? (Motivare).

SECTION 2: OS161

Domanda 4 (Sincronizzazione Produttore/Consumatore)

Contesto: Si considerino tre kernel thread in OS161, i quali implementano un'attività di trasferimento dati basata sul modello produttore/consumatore (due produttori, più consumatori). I thread condividono una struttura C di tipo `struct prodCons` definita come segue:

```
#define NumP 2
struct prodCons {
    void *data[NumP];
    int size[NumP];
    int busy[NumP];
    int ready[NumP];
    struct lock *pc_lk;
    struct cv *pc_cv;
    /* other stuff omitted */
};
```

Il produttore i (con $i = 0$ oppure 1) aggiorna le i -esime entry nei vettori `data` e `size` (dato e dimensione, in pratica è un buffer). I consumatori possono leggere indifferentemente una delle due caselle dei vettori. Il lock viene utilizzato per la mutua esclusione sui vettori condivisi. I vettori di flag `ready` e `busy` indicano, se valgono 1:

- `ready`: dato (e dimensione) corrispondente pronto per essere letto
- `busy`: dato attualmente in lettura o in scrittura

Prima di lavorare sui dati, un consumatore deve quindi attendere/verificare che una opportuna condizione sui flag sia vera. Questo avviene chiamando la funzione: `int consumerWait(struct prodCons *pc);`

Si risponda alle seguenti domande:

-) La struttura condivisa può essere posizionata nello stack di thread o dovrebbe essere una variabile globale o altro?
-) Dato che i campi `pc_lk` e `pc_cv` sono puntatori, dove dovrebbero essere chiamate `lock_create()` e `cv_create()`? Nei thread produttori, nel thread consumatore? Altrove?
-) Si fornisca un'implementazione della funzione `consumerWait`. Non è necessario alcun codice produttore e/o consumatore, solo la funzione, tenendo presente che la funzione ha come scopo ritornare l'indice (0 o 1) del buffer da cui leggere (avendone posto a 1 il corrispondente flag `busy`/letto). Il dato non va letto nella funzione.

Domanda 5 (Tabelle File di Processo e di Sistema)

Contesto: Si consideri in OS161 una possibile implementazione delle system call relative al file system, basate su una tabella di processo e su una di sistema, definite come segue:

```
/* system open file table */
struct openfile {
    struct vnode *vn;
    off_t offset;
    unsigned int countRef;
```

```
};

/* this is a global variable */
struct openfile systemFileTable[SYSTEM_OPEN_MAX];

/* user open file table: this is a field of struct proc */
struct openfile *fileTable[OPEN_MAX];
```

Si risponda alle domande seguenti (motivando le risposte):

-) Perché `systemFileTable` è una variabile globale, mentre `fileTable` è un campo di `struct proc`?
-) Si supponga che, dati due processi user P1 e P2 (con `p1` e `p2` puntatori alle rispettive `struct proc`) siano vere le seguenti uguaglianze:
 - `p1->fileTable[5] == &systemFileTable[20]`
 - `p2->fileTable[8] == &systemFileTable[20]`

A quale/i file descriptor (fd) corrisponde/corrispondono le parti di tabella visualizzate? 5, 8, 20? (Motivare). Se P1 e P2 fanno solo letture sul/sui file di cui sopra, le letture sono indipendenti oppure c'è interazione tra i processi?

-) Si supponga di aver implementato il supporto per la system call `SYS_lseek`. Una chiamata a `lseek` effettuata da P1 (si veda domanda B) influenzerà la successiva `read` effettuata da P2:
 1. sempre?
 2. mai?
 3. solo se il parametro offset è `SEEK_CUR`?

(sono possibili selezioni multiple, per ogni selezione occorre motivare la risposta).

ANSWERS

Domanda 1 (Memoria Virtuale, TLB e EAT)

A) Calcolo del TLB miss ratio (Lower/Upper Bound):

Dati:

- $T_{\text{RAM}} = 75 \text{ ns}$.
- Tempo di accesso TLB trascurabile (0 ns).
- Page table a 2 livelli $\implies$ in caso di miss, la traduzione costa 2 accessi in RAM aggiuntivi prima di leggere il dato.
- $EAT \geq 100 \text{ ns}$.

Sia m il TLB miss ratio. Il tempo effettivo di accesso (EAT) è dato dalla somma tra il tempo medio di traduzione e il tempo di accesso al dato:

$$EAT = \text{Tempo di traduzione} + \text{Tempo di accesso al dato}$$

Il tempo di traduzione è 0 in caso di TLB hit, e costa $2 \times T_{\text{RAM}}$ in caso di TLB miss. Il tempo di accesso al dato è sempre $1 \times T_{\text{RAM}}$.

$$EAT = (m \times 2 \times T_{\text{RAM}}) + T_{\text{RAM}}$$

Sostituendo i valori forniti dalla traccia:

$$EAT = (m \times 2 \times 75) + 75 = 150m + 75$$

Imponendo la condizione della traccia $EAT \geq 100$:

$$150m + 75 \geq 100$$

$$150m \geq 25$$

$$m \geq \frac{25}{150}$$

$$m \geq \frac{1}{6}$$

Il TLB miss ratio deve essere pari o superiore a $1/6$ (circa 16,67%). Essendo questo il valore minimo affinché si verifichi la condizione di degrado indicata, si tratta di un **lower bound** (limite inferiore).

B) Intervallo di valori per la frequenza di Page Fault (P_{PF}):

Dati:

- $EAT_{\text{base}} = 100 \text{ ns}$.
- $T_{\text{PF}} = 2 \text{ ms} = 2.000.000 \text{ ns}$.

- Degrado accettabile: tra il 10% ($EAT_{\min} = 110$ ns) e il 30% ($EAT_{\max} = 130$ ns).

Utilizzando la formula completa: $EAT_{PF} = EAT_{base} + P_{PF} \times T_{PF}$ (approssimazione tollerata essendo $T_{PF} \gg EAT_{base}$).

1. Caso 10%:

$$110 = 100 + P_{PF} \times 2.000.000 \implies P_{PF} = \frac{10}{2.000.000} = 5 \times 10^{-6} \text{ (0,0005\%)}$$

2. Caso 30%:

$$130 = 100 + P_{PF} \times 2.000.000 \implies P_{PF} = \frac{30}{2.000.000} = 1,5 \times 10^{-5} \text{ (0,0015\%)}$$

L'intervallo compatibile è quindi: $0,0005\% \leq P_{PF} \leq 0,0015\%$.

C) Calcolo probabilità p_1 e p_2 su due stringhe di riferimento:

Innanzitutto, si calcola la frequenza di Page Fault specifica per ciascuna stringa isolata:

$$f_1 = \frac{PF_1}{len(w_1)} = \frac{100}{1.000.000} = 0,00010$$

$$f_2 = \frac{PF_2}{len(w_2)} = \frac{300}{2.000.000} = 0,00015$$

La frequenza globale misurata nel sistema ($f = 0,00012$) rappresenta il valore atteso (media ponderata) delle due frequenze. Si imposta quindi il seguente sistema lineare a due incognite:

$$\begin{cases} p_1 + p_2 = 1 & \text{(La somma delle probabilità deve essere 100\%)} \\ f = p_1 \cdot f_1 + p_2 \cdot f_2 & \text{(Valore atteso globale)} \end{cases}$$

Sostituendo i valori noti: $\begin{cases} p_2 = 1 - p_1 \\ 0,00012 = p_1(0,00010) + p_2(0,00015) \end{cases}$

Si sostituisce la prima equazione nella seconda:

$$0,00012 = 0,00010p_1 + 0,00015(1 - p_1)$$

$$0,00012 = 0,00010p_1 + 0,00015 - 0,00015p_1$$

$$-0,00003 = -0,00005p_1$$

$$p_1 = \frac{0,00003}{0,00005} = \frac{3}{5} = 0,6$$

Ricavando p_2 :

$$p_2 = 1 - 0,6 = 0,4$$

Le probabilità richieste sono quindi $p_1 = 60\%$ e $p_2 = 40\%$.

Domanda 2 (File System Unix-like e Inodes)

Dati di Base:

- Dimensione Blocco = 2 KB = 2048 Byte.
- Dimensione Puntatore = 32 bit = 4 Byte.
- Puntatori per blocco indice = $2048/4 = 512$ puntatori.
- Capacità Disco = 800 GB = 800×2^{30} Byte.

- Blocchi fisici totali = $\frac{800 \times 2^{30}}{2048} = 400 \times 2^{20} = \mathbf{419.430.400}$ blocchi.

A) Numero massimo di file ospitabili (N2 e N3):

Per massimizzare il numero di file, occorre calcolare lo spazio minimo (in blocchi) affinché un file attivi l'indirizzione doppia (N2) o tripla (N3). Lo spazio occupato da un file è la somma dei blocchi dato e dei blocchi indice necessari (assumendo le dimensioni degli inode e altri metadati trascurabili).

1. **Indicizzazione Indiretta Doppia (N2):** Il file deve esaurire i blocchi diretti (10) e singoli indiretti (512) per utilizzare il 1° blocco della doppia indirizzione.

- Blocchi dato minimi: $10 + 512 + 1 = \mathbf{523}$.
- Blocchi indice minimi: 1 (singolo) +1 (doppio livello esterno) +1 (doppio livello interno) = **3**.
- Spazio totale minimo per file N2: $523 + 3 = \mathbf{526}$ blocchi.

$$\text{Numero Massimo File (N2)} = \left\lfloor \frac{419.430.400}{526} \right\rfloor = \mathbf{797.396} \text{ file.}$$

2. **Indicizzazione Indiretta Tripla (N3):** Il file deve esaurire i blocchi diretti, i singoli e tutti i doppi indiretti ($512 \times 512 = 262.144$) per attivare la tripla indirizzione.

- Blocchi dato minimi: $10 + 512 + 262.144 + 1 = \mathbf{262.667}$.
- Blocchi indice minimi: 1 (singolo) +1 + 512 (doppi) +1 + 1 + 1 (i 3 livelli del triplo) = **517**.
- Spazio totale minimo per file N3: $262.667 + 517 = \mathbf{263.184}$ blocchi.

$$\text{Numero Massimo File (N3)} = \left\lfloor \frac{419.430.400}{263.184} \right\rfloor = \mathbf{1.593} \text{ file.}$$

B) Calcolo esatto dei blocchi Dato e Indice per file di 10240.5 KB:

- **Blocchi Dato:** $\lceil \frac{10240.5 \text{ KB}}{2 \text{ KB}} \rceil = \mathbf{5121}$ blocchi dato.
- **Blocchi Indice:** I 5121 blocchi sono così mappati:
 - I primi 10 sono diretti (0 blocchi indice).
 - I successivi 512 richiedono **1** blocco singolo indiretto.
 - I restanti $5121 - 10 - 512 = 4599$ blocchi ricadono nell'indirizzione doppia.
 - Livello interno del doppio indiretto: $\lceil \frac{4599}{512} \rceil = \mathbf{9}$ blocchi indice interni.
 - Livello esterno del doppio indiretto: **1** blocco indice esterno (che punta ai 9 interni).

Totale Blocchi Indice = $1 + 9 + 1 = \mathbf{11}$ blocchi indice.

C) Operazione lseek con offset 0x00800000 da SEEK_END:

Secondo lo standard POSIX, un offset positivo applicato a `SEEK_END` sposta il cursore *in avanti* oltre la fine del file, espandendone la portata logica. (Qualora il docente intendesse uno spostamento a ritroso ignorando l'assenza del segno meno, l'offset logico sarebbe $10.486.272 - 8.388.608 = 2.097.664$, corrispondente al blocco 1024). Applichiamo l'interpretazione rigorosa in avanti.

1. **Numero del Blocco Logico:**

Dimensione File in Byte = $10240,5 \times 1024 = 10.486.272$ B.

Offset = $0x00800000$ (esa) = $8.388.608$ B (decimale).

Nuova posizione byte = $10.486.272 + 8.388.608 = 18.874.880$ B.

Il blocco logico (numerato da 0) è: $\lfloor \frac{18.874.880}{2048} \rfloor = \mathbf{9216}$.

2. **Riga del blocco di indici esterno:**

Il blocco logico 9216 ricade nell'indirizzione doppia. Si sottraggono le mappature dirette e singole per ottenere l'indice relativo all'area dell'indirizzione doppia:

$$\text{Indice Relativo} = 9216 - 10 - 512 = \mathbf{8694}.$$

Il blocco di indici **esterno** mappa intervalli di 512 blocchi per volta. La riga che contiene il puntatore al blocco interno corretto si calcola dividendo l'indice relativo per 512:

$$\text{Riga (Indice Array)} = \lfloor \frac{8694}{512} \rfloor = \mathbf{16}.$$

Il puntatore risiede pertanto alla **riga 16** (la diciassettesima casella, essendo l'indice 0-based) del blocco doppio indiretto principale.

Domanda 3 (Dispositivi di I/O, Polling e Interrupts)

A) Effettuare polling diretto dai registri da un processo user:

NON è possibile.

Motivazione: L'accesso ai registri di stato e di controllo di un dispositivo hardware (sia tramite porte I/O dedicate, sia tramite Memory-Mapped I/O) richiede istruzioni privilegiate e l'accesso a porzioni di memoria fisica riservate al kernel. Un programma in esecuzione in *User Mode* non dispone dei permessi hardware necessari. Qualsiasi tentativo di leggere o scrivere direttamente tali registri scatenerà una *Trap* (violazione di privilegio o Segmentation Fault) da parte della MMU, portando alla terminazione immediata del processo da parte del Sistema Operativo.

B) Generazione interrupt per tastiera (Carattere vs Invio):

Ci si aspetta la generazione di un interrupt **per ogni singolo carattere** ricevuto.

Motivazione: La tastiera è un *character device* interattivo che richiede latenze microscopiche per due motivi fondamentali:

1. **Gestione dell'Echo:** L'utente necessita di un feedback visivo immediato (il carattere digitato deve apparire subito a schermo). Se l'interrupt avvenisse solo all'invio, l'utente scriverebbe "alla cieca" e il driver terminale non potrebbe gestire interattivamente il tasto Backspace (cancellazione in tempo reale).
2. **Caratteri di Controllo (Segnali):** Il sistema operativo deve poter intercettare immediatamente sequenze critiche (come **Ctrl+C** per inviare SIGINT, o **Ctrl+Z** per SIGTSTP). Attendere un carattere di fine riga renderebbe impossibile terminare tempestivamente un processo utente in loop infinito.

C) Scrittura con più coppie puntatore/dimensione:

SI, è possibile. Questa tecnica prende il nome di **Scatter-Gather I/O** (o *Vectored I/O*).

Motivazione: Quando un processo possiede dati logicamente contigui ma fisicamente sparsi in aree di memoria RAM distinte (es. header, payload e footer allocati separatamente), può utilizzare chiamate di sistema vettoriali (come **writenv()** in POSIX). Il processo passa al kernel un array di strutture (es. **iovec**), ciascuna contenente un puntatore e una dimensione. Un controller DMA avanzato che supporta Scatter-Gather leggerà autonomamente dai vari buffer sparsi in memoria e trasferirà i dati al dispositivo di blocco in un'unica operazione di output

continua, evitando al processo il costoso overhead di fare *memory copy* in un unico mega-buffer prima di chiamare l'I/O.

D) Riordino delle operazioni di I/O (I/O Scheduling):

SI, è possibile che vengano riordinate, in entrambi i casi.

Motivazione:

- **Da un altro processo:** Le operazioni di I/O dirette a un dispositivo a blocchi (disco) vengono inserite dal kernel in una *Device Queue*. L'**I/O Scheduler** del sistema operativo non estrae le richieste con logica FIFO stretta, ma applica algoritmi di riordino spaziale (come SCAN/Elevator o C-SCAN) per fondere richieste adiacenti e minimizzare il costoso movimento meccanico della testina (*Seek Time*).
- **Dallo stesso processo:** Anche le richieste del medesimo processo possono subire riordino se il processo utilizza chiamate di **I/O asincrono (AIO)**. Poiché la system call ritorna immediatamente senza bloccare, il processo può inviare una "raffica" di richieste di lettura/scrittura. Il SO (o il controller hardware del disco stesso, tramite tecnologie come NCQ - Native Command Queuing) elaborerà queste richieste nell'ordine fisicamente più vantaggioso, e non in quello di arrivo temporale.

Domanda 4 (Sincronizzazione Produttore/Consumatore)

A) Posizionamento della struttura condivisa:

La struttura **NON** può essere posizionata nello stack di un thread.

Motivazione: Lo stack è un'area di memoria strettamente privata e locale per ciascun thread, utilizzata per i record di attivazione delle funzioni e le variabili locali. Un thread non può (e non deve) accedere in modo sicuro allo stack di un altro thread. Affinché la struttura **struct prodCons** sia condivisa, deve essere allocata in un'area di memoria comune a tutti i thread del kernel, ovvero come **variabile globale** (nel segmento **.bss/.data** del kernel) oppure allocata dinamicamente nell'**heap del kernel** (utilizzando **kmalloc**) e passata come puntatore ai thread al momento della loro creazione.

B) Creazione delle primitive di sincronizzazione (Lock e CV):

I campi **pc_lk** e **pc_cv** sono primitive del kernel. Le funzioni **lock_create()** e **cv_create()** devono essere chiamate **altrove**, specificamente dal **thread principale (main/parent thread)** che si occupa del setup del sistema, **prima** che i thread produttori e consumatori vengano effettivamente creati e avviati.

Motivazione: Se l'inizializzazione venisse demandata a un produttore o a un consumatore, si creerebbe una gravissima *race condition*. Un thread potrebbe tentare di acquisire il lock (**lock_acquire**) prima che un altro thread abbia avuto il tempo CPU di eseguire la **lock_create()**, causando una dereferenziazione di puntatore a NULL e un conseguente *Kernel Panic*.

C) Implementazione della funzione consumerWait:

L'implementazione richiede l'uso del costrutto **while** associato alla Condition Variable per gestire l'attesa senza *busy waiting*, in conformità con gli standard di OS161.

```
int consumerWait(struct prodCons *pc) {
    int index;

    /* Acquisizione della mutua esclusione sulla struttura */
    lock_acquire(pc->pc_lk);

    /* CONDIZIONE DI ATTESA:
     * Il consumatore si addormenta finché NON c'è almeno un buffer
```

```

    * che sia 'ready' (1) E contemporaneamente NON 'busy' (0).
    */
while (!((pc->ready[0] == 1 && pc->busy[0] == 0) ||
        (pc->ready[1] == 1 && pc->busy[1] == 0))) {

    /* Rilascia atomicamente il lock e si mette in attesa sulla CV */
    cv_wait(pc->pc_cv, pc->pc_lk);
}

/* Usciti dal while, abbiamo la garanzia che almeno un buffer è pronto.
 * Identifichiamo quale dei due (dando precedenza allo 0) */
if (pc->ready[0] == 1 && pc->busy[0] == 0) {
    index = 0;
} else {
    index = 1;
}

/* Impostiamo il flag busy a 1 come richiesto dalla traccia,
 * per indicare che stiamo per leggere il dato */
pc->busy[index] = 1;

/* Rilascio della mutua esclusione */
lock_release(pc->pc_lk);

return index;
}

```

Domanda 5 (Tabelle File di Processo e di Sistema)

A) Variabile globale systemFileTable vs campo locale fileTable:

Motivazione: In ambiente UNIX/OS161, si utilizzano due livelli di astrazione:

- **fileTable (Processo):** È l'array locale al singolo processo. L'indice di questo array corrisponde al **File Descriptor (fd)** visto dallo spazio utente (es. 0, 1, 2). Mantenere questa tabella privata permette a ogni processo di avere il proprio spazio isolato di fd (il fd 5 di P1 e il fd 5 di P2 sono indipendenti logici).
- **systemFileTable (Globale):** È la tabella di sistema che memorizza lo stato effettivo dell'"apertura" di un file, in particolare la posizione corrente di lettura/scrittura (**offset**) e il reference count (**countRef**). Essendo globale, permette a più processi (es. padre e figlio dopo una **fork**) di condividere la stessa istanza di file aperto, mantenendo sincronizzati gli offset.

B) Identificazione dei File Descriptor e Interazione delle letture:

- **File Descriptor (fd):** I file descriptor sono gli indici della tabella locale del processo. Pertanto, i fd associati a queste aperture sono **5 per P1** e **8 per P2**. Il numero 20 è unicamente un indice interno alla memoria del kernel della tabella di sistema e non è visibile nello spazio utente.
- **Interazione tra i processi:** Le letture **NON sono indipendenti; vi è forte interazione**. Poiché entrambi i puntatori locali indirizzano la medesima struttura in memoria (&systemFileTable[20]), i due processi condividono la stessa istanza della **struct**

`openfile`. Poiché il campo `offset` si trova all'interno di questa struttura condivisa, ogni operazione di `read()` effettuata da P1 farà avanzare l'offset globale; di conseguenza, una successiva `read()` di P2 inizierà a leggere dal punto in cui P1 si è fermato.

C) Effetto di `SYS_lseek` effettuata da P1 sulle letture di P2:

- **1. sempre? VERO.**

Motivazione: Poiché P1 e P2 condividono il puntatore alla medesima entry `systemFileTable[20]`, condividono fisicamente lo stesso campo `offset`. Qualunque operazione di `lseek` eseguita da P1 aggiornerà il valore di quell'offset condiviso. La successiva chiamata a `read()` da parte di P2 utilizzerà l'offset appena modificato, subendone sempre l'influenza, indipendentemente dal parametro originale della seek.

- **2. mai? FALSO.** (Vedi motivazione 1).

- **3. solo se il parametro `offset` è `SEEK_CUR`? FALSO.**

Motivazione: L'interazione avviene indipendentemente dalla modalità della `lseek`. Che P1 sposti il cursore in modo assoluto (`SEEK_SET`), relativo (`SEEK_CUR`) o dalla fine del file (`SEEK_END`), l'effetto finale è la scrittura di un nuovo valore nel campo `offset` condiviso, che influenzerà sempre P2.